LAPORAN PRAKTIKUM
ALGORITMA DAN PEMROGRAMAN 1
MODUL 1
“TIPE DATA & VARIABEL”

DISUSUN OLEH:

Alvin Setya Wardana

109082500107

S1 IF-13-02

DOSEN:

Yohani Setiya Rafika Nur, M. Kom.

PROGRAM STUDI S1 TEKNIK INFORMATIKA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2024/2025

DASAR TEORI

1. **Tipe Data dan Variabel** Tipe data adalah klasifikasi nilai yang menentukan bagaimana data disimpan dan operasi apa yang dapat dilakukan terhadapnya. Dalam bahasa Go, tipe data dasar meliputi int, float64, string, dan bool. Variabel adalah wadah untuk menyimpan nilai dengan tipe tertentu, yang dapat digunakan dan dimodifikasi selama program berjalan. Pemahaman tipe data dan variabel menjadi dasar dalam pengolahan data dan logika program.

2. **Rekursi dalam Pemrograman** Rekursi adalah teknik pemrograman di mana sebuah fungsi memanggil dirinya sendiri untuk menyelesaikan masalah yang lebih kecil dari masalah semula. Rekursi biasanya memiliki dua bagian penting: kondisi dasar (base case) untuk menghentikan pemanggilan, dan langkah rekursif (recursive step) untuk memecah masalah. Rekursi banyak digunakan dalam perhitungan matematis seperti deret Fibonacci, faktorial, dan pencarian pola.

3. **Deret Fibonacci** Deret Fibonacci adalah barisan bilangan yang dimulai dari 0 dan 1, dengan setiap suku berikutnya merupakan hasil penjumlahan dua suku sebelumnya. Rumus umum:

$$S_n = S_{n-1} + S_{n-2}$$

Deret ini sering digunakan untuk melatih pemahaman rekursi karena strukturnya yang sederhana namun efektif.

4. **Pola Bintang dengan Rekursi** Pola bintang dapat dicetak menggunakan rekursi dengan cara memanggil fungsi untuk mencetak baris sebelumnya, lalu mencetak baris saat ini. Konsep ini menunjukkan bagaimana rekursi dapat digunakan untuk membangun pola bertingkat.

5. **Faktor Bilangan** Faktor dari suatu bilangan Nadalah bilangan bulat positif yang dapat membagi Ntanpa sisa. Dengan rekursi, faktor dapat dicetak dengan memeriksa setiap bilangan dari 1 hingga N.

6. **Deret Simetris (Turun-Naik)** Rekursi dapat digunakan untuk mencetak deret menurun dari Nke 1, lalu naik kembali ke N. Teknik ini memanfaatkan sifat *backtracking* dari rekursi.

7. **Bilangan Ganjil dengan Rekursi** Rekursi juga dapat digunakan untuk menampilkan bilangan ganjil dari 1 hingga Ndengan memeriksa kondisi i % 2 != 0.

8. **Perpangkatan dengan Rekursi** Operasi pangkat x^ydapat dihitung dengan rekursi menggunakan rumus:

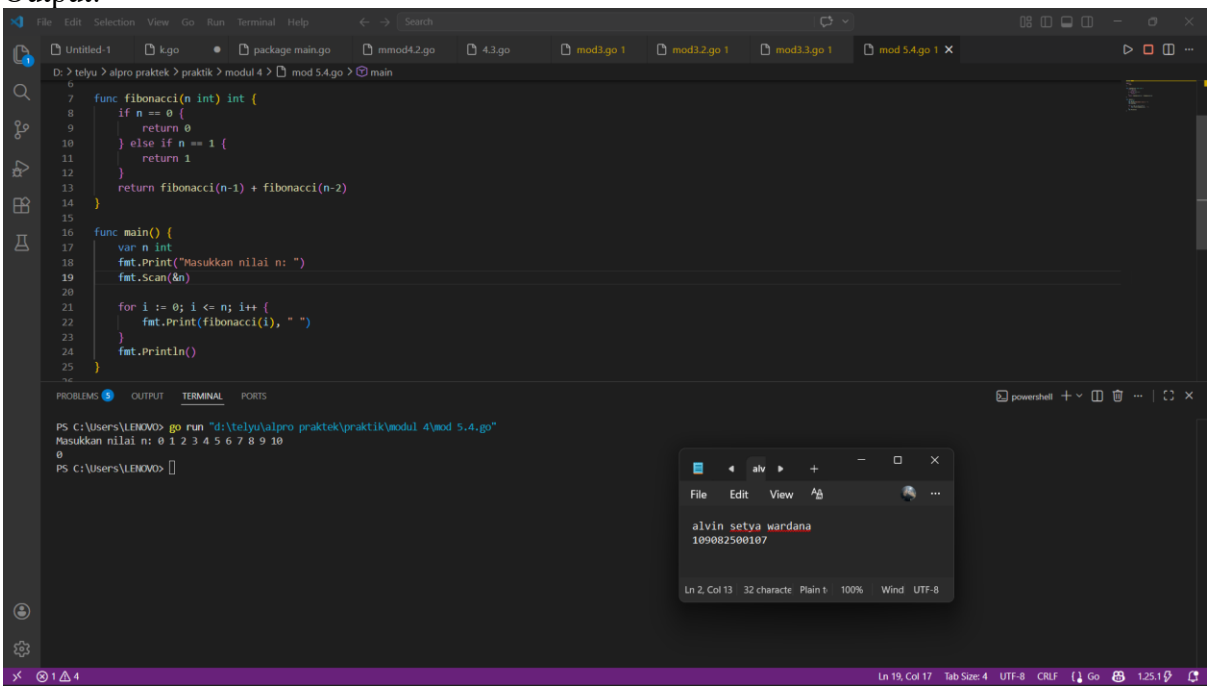
CONTOH SOAL

1. Latihan1

Source Code:

```
package main

import (
    "fmt"
)

func fibonacci(n int) int {
    if n == 0 {
        return 0
    } else if n == 1 {
        return 1
    }
    return fibonacci(n-1) + fibonacci(n-2)
}

func main() {
    var n int
    fmt.Print("Masukkan nilai n: ")
    fmt.Scan(&n)

    for i := 0; i <= n; i++ {
        fmt.Print(fibonacci(i), " ")
    }
    fmt.Println()
}
```

Output:



Deskripsi Program:

Program ini mengimplementasikan deret Fibonacci menggunakan fungsi rekursif dalam bahasa Go. Fungsi fibonacci(n) akan memanggil dirinya sendiri untuk menghitung nilai suku ke-n berdasarkan rumus $S_n = S_{n-1} + S_{n-2}$. Program menerima sebuah bilangan bulat n sebagai masukan, lalu mencetak deret Fibonacci dari suku ke-0 hingga suku ke-n.

SOAL LATIHAN

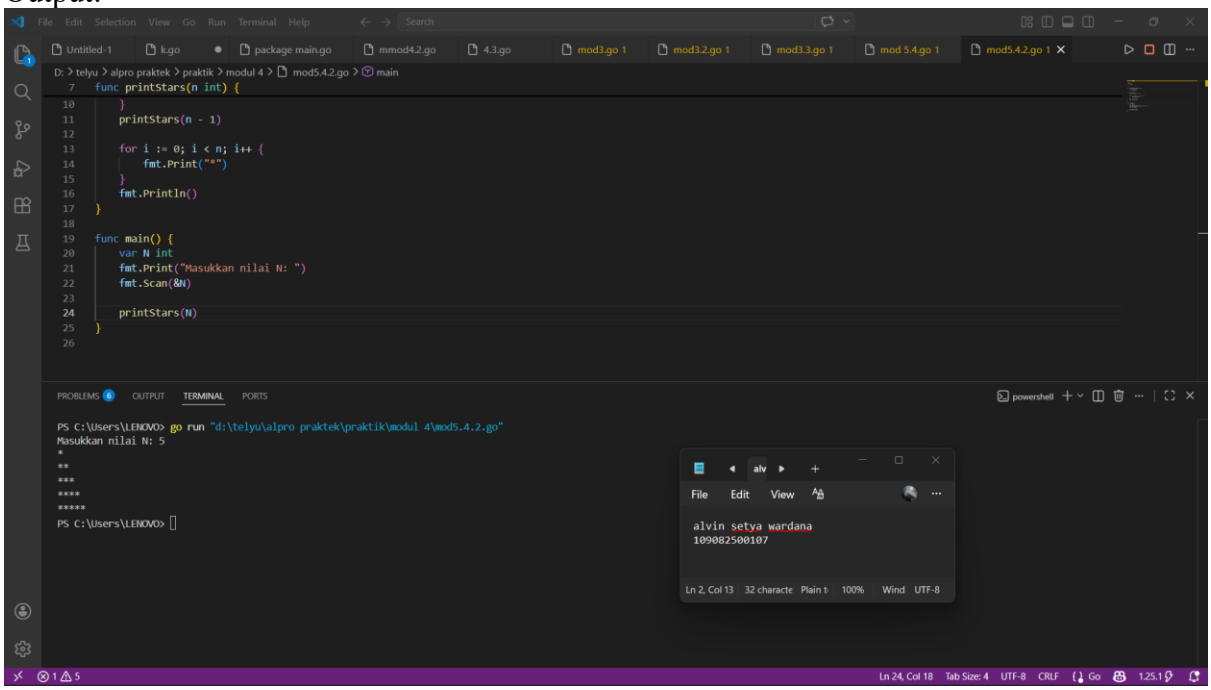
Statement perulangan

1.

Source Code:

```
package main

import (
    "fmt"
)

func printStars(n int) {
    if n == 0 {
        return
    }
    printStars(n - 1)

    for i := 0; i < n; i++ {
        fmt.Print("*")
    }
    fmt.Println()
}

func main() {
    var N int
    fmt.Print("Masukkan nilai N: ")
    fmt.Scan(&N)

    printStars(N)
}
```

Output:



Deskripsi Program:

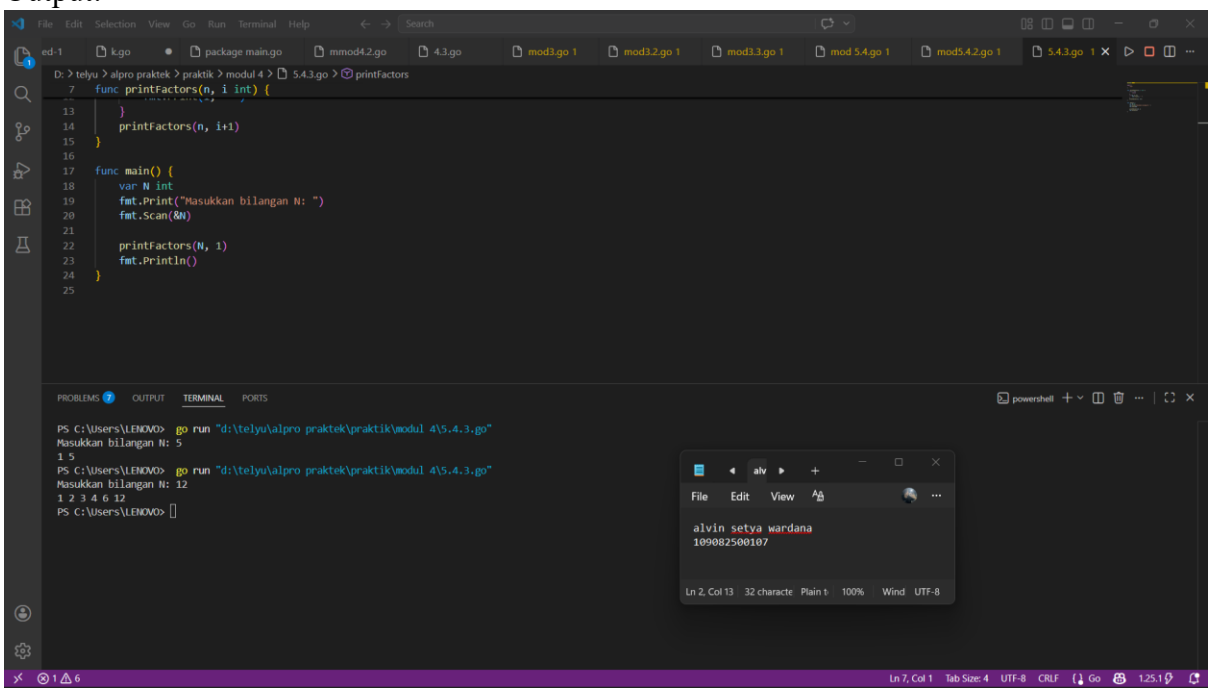
Program ini menggunakan fungsi rekursif printStars(n) untuk mencetak pola bintang. Fungsi akan memanggil dirinya sendiri dengan nilai n-1 hingga mencapai kondisi dasar (n == 0). Setelah itu, setiap pemanggilan akan mencetak baris bintang sesuai nilai n. Dengan cara ini, pola bintang dari 1 hingga N baris terbentuk secara bertahap.

Source Code:

```
package main

import (
    "fmt"
)

func printFactors(n, i int) {
    if i > n {
        return
    }
    if n%i == 0 {
        fmt.Print(i, " ")
    }
    printFactors(n, i+1)
}
```

```
func main() {
    var N int
    fmt.Print("Masukkan bilangan N: ")
    fmt.Scan(&N)

    printFactors(N, 1)
    fmt.Println()
}
```

Output:



Deskripsi Program:

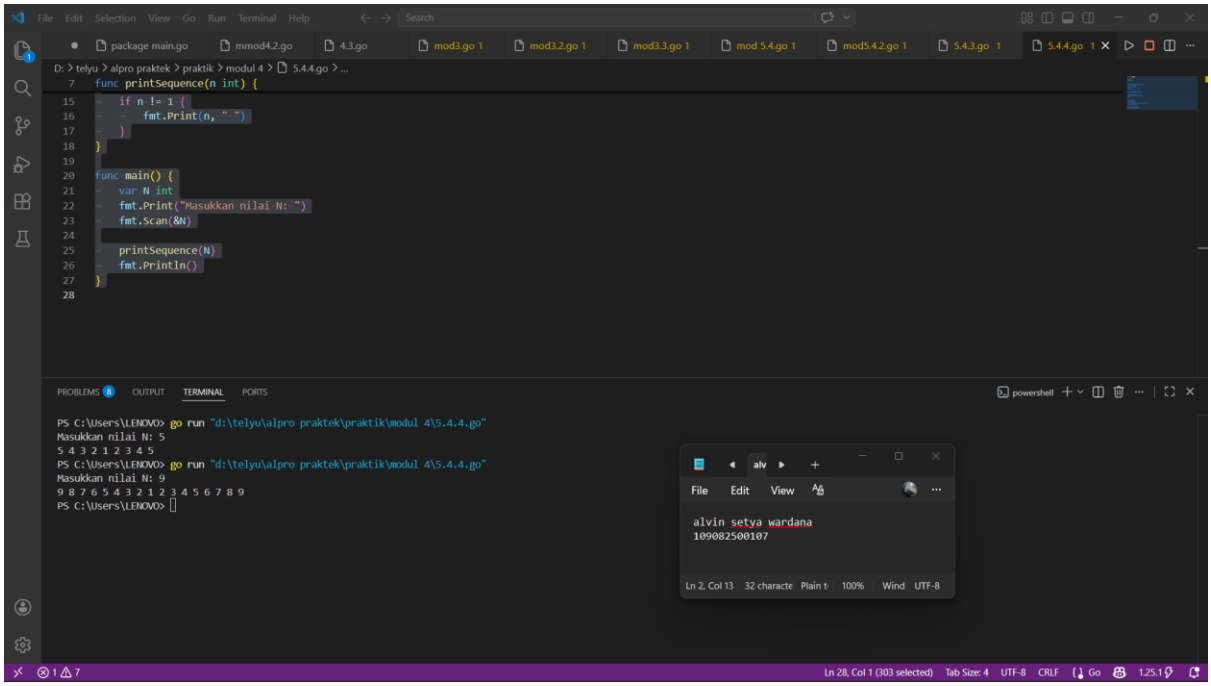
Program ini menerima sebuah bilangan bulat positif N sebagai masukan. Fungsi rekursif printFactors(n, i) digunakan untuk mengecek setiap bilangan dari 1 hingga N. Jika i adalah faktor dari N (yaitu N % i == 0), maka i dicetak. Fungsi akan terus dipanggil dengan nilai i+1 hingga mencapai N. Dengan cara ini, semua faktor dari N ditampilkan secara terurut.

Source Code:

```
package main


import (
```

```
    "fmt"
)

func printSequence(n int) {
    if n == 0 {
        return
    }
    fmt.Print(n, " ")

    printSequence(n - 1)

    if n != 1 {
        fmt.Print(n, " ")
    }
}

func main() {
    var N int
    fmt.Print("Masukkan nilai N: ")
    fmt.Scan(&N)

    printSequence(N)
    fmt.Println()
}
```

Output:



Deskripsi Program:

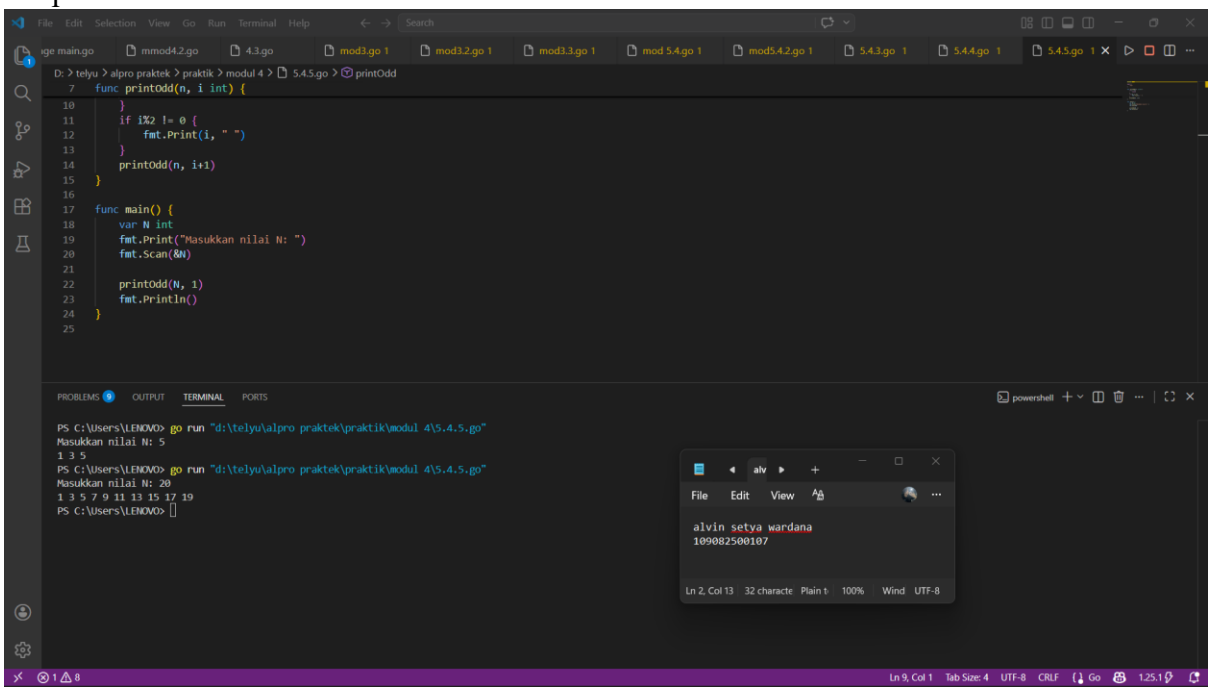
Program ini menerima sebuah bilangan bulat positif N sebagai masukan. Fungsi rekursif printSequence(n) digunakan untuk mencetak deret menurun dari N ke 1. Setelah mencapai kondisi dasar (n == 0), fungsi kembali (*backtracking*) dan mencetak nilai naik kembali hingga N. Dengan cara ini, terbentuk pola deret simetris dari N ke 1 lalu kembali ke N.

Source Code:

```
package main


import (
    "fmt"
)


func printOdd(n, i int) {
    if i > n {
        return
    }
    if i%2 != 0 {
        fmt.Print(i, " ")
    }
    printOdd(n, i+1)
```

```
}

func main() {
    var N int
    fmt.Print("Masukkan nilai N: ")
    fmt.Scan(&N)

    printOdd(N, 1)
    fmt.Println()
}
```

Output:



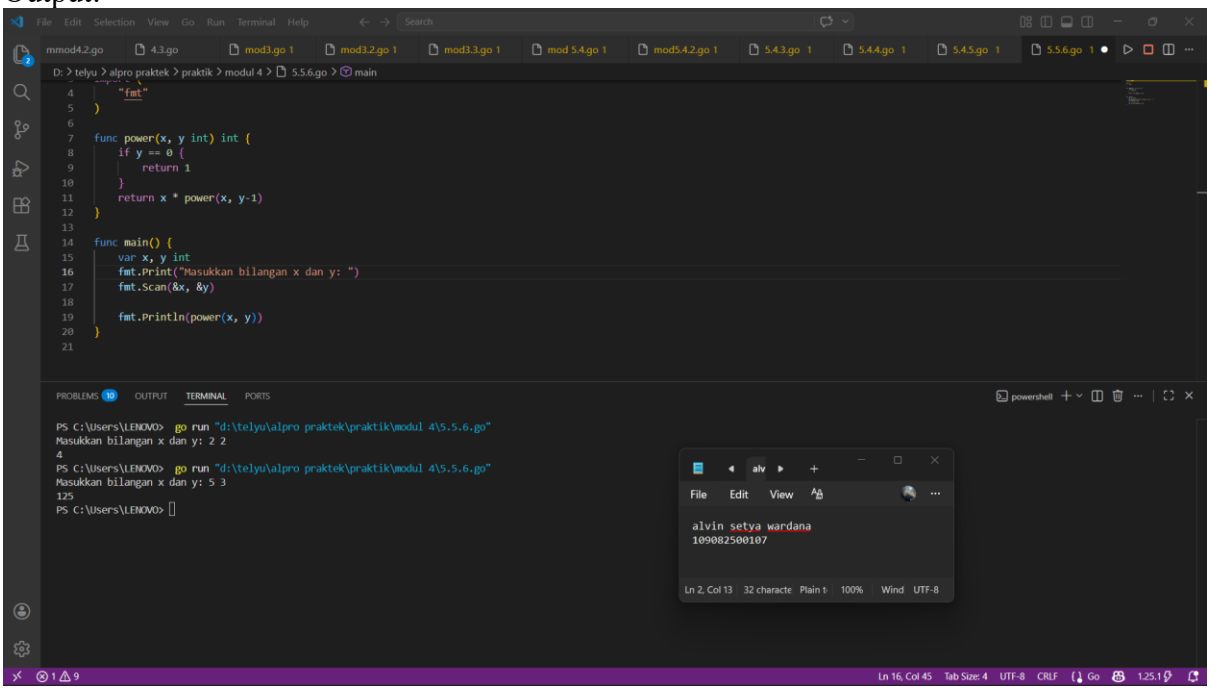
Deskripsi Program:

Program ini menerima sebuah bilangan bulat positif N sebagai masukan. Fungsi rekursif printOdd(n, i) digunakan untuk mengecek setiap bilangan dari 1 hingga N. Jika bilangan tersebut ganjil, maka dicetak. Fungsi akan terus dipanggil dengan nilai i+1 hingga mencapai N. Dengan cara ini, semua bilangan ganjil dari 1 sampai N ditampilkan secara terurut.

Source Code:

```
package main


import (
    "fmt"
)
func power(x, y int) int {
    if y == 0 {
        return 1
    }
    return x * power(x, y-1)


func main() {
    var x, y int
    fmt.Print("Masukkan bilangan x dan y: ")
    fmt.Scan(&x, &y)
    fmt.Println(power(x, y))
}
.Println()
}
```

Output:



Deskripsi Program:

Program ini menerima dua bilangan bulat x dan y sebagai masukan. Fungsi rekursif power(x, y) digunakan untuk menghitung hasil perpangkatan. Jika y bernilai 0, fungsi mengembalikan 1 sebagai kondisi dasar. Jika y lebih besar dari 0, fungsi akan memanggil dirinya sendiri dengan y-1 dan mengalikan hasilnya dengan x. Dengan cara ini, hasil x^y diperoleh tanpa menggunakan pustaka matematika bawaan.

DAFTAR PUSTAKA

- Munir, Rinaldi. *Matematika Diskrit*. Bandung: Informatika, 2016.
- Kurniawan, Budi Raharjo. *Algoritma dan Pemrograman dengan Go*. Bandung: Informatika, 2020.
- Donovan, Alan A. A., dan Kernighan, Brian W. *The Go Programming Language*. Addison-Wesley, 2015.
- Wirth, Niklaus. *Algorithms + Data Structures = Programs*. Prentice Hall, 1976.
- Sutarman. *Pengantar Teknologi Informasi*. Jakarta: Bumi Aksara, 2012.
- Pressman, Roger S. *Software Engineering: A Practitioner's Approach*. McGraw-Hill, 2010.